Spark vs. Ray: The Data Engineering Duel

Distributed ETL Benchmark on NYC Yellow Taxi Data

DA3408 Assignment 3

Rishikesh

DA24B050

October 2026

Overview

This assignment compares Apache Spark and Ray on the same data-cleaning problem: processing the NYC Yellow Taxi trip data from January 2024 through August 2026. The main question is simple—which framework handles this ETL workload more efficiently when the pipeline includes both ordinary dataframe operations and a Python-based transformation?

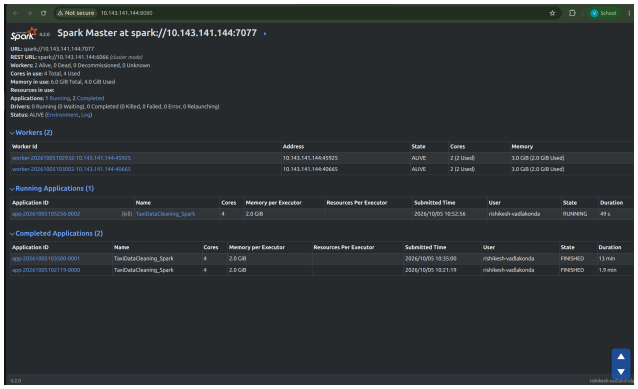
The benchmark uses a distributed setup with one coordinator and two workers. For both frameworks, the logical pipeline consists of reading the monthly Parquet files, removing nulls and duplicates, converting timestamps, joining trip records with the taxi-zone lookup, computing average speed, and writing the cleaned result back to Parquet. The assignment asks for total execution time, peak CPU and memory usage, and a separate measurement of the Python UDF stage.

All performance numbers reported here come from the completed benchmark runs and the corresponding `top` recordings. No synthetic benchmark values are used.

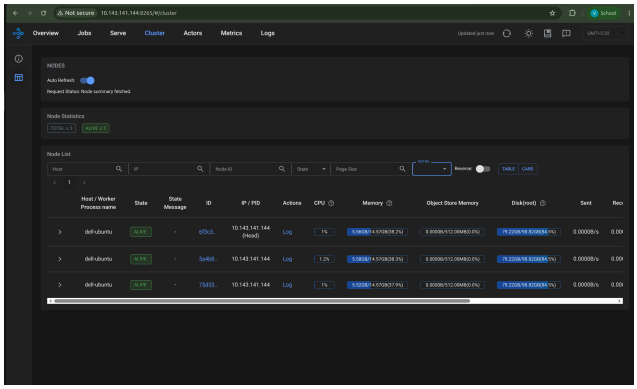
Cluster Setup

Spark cluster

The Spark deployment used one Spark Master and two Spark Workers. The application connected to the master at `spark://10.143.141.144:7077`. The Spark application was configured for two executors with two cores each, giving four worker cores in total. The workers were started manually before launching the Python application.



(a) Spark Master.



(b) Ray Dashboard.

Figure 1: Cluster evidence for the two distributed implementations.

Ray cluster

The Ray deployment used one Ray Head node and two Ray Workers. The head node acted as the coordinator, while each worker advertised two CPUs. The application connected to the already-running cluster through `ray.init(address="auto")`; Ray Data then scheduled the dataset operations across the available worker resources. The head node was started with zero compute CPUs for normal worker tasks, leaving the four advertised CPUs on the two workers.

Data Pipeline

Both implementations follow the same intended sequence of operations:

Read Parquet files → Remove nulls → Remove duplicates → Format timestamps
→ Join with taxi-zone lookup → Calculate average speed → Write Parquet

In Spark, null removal and duplicate removal are performed with dataframe operations. The taxi-zone table is small, so the join uses Spark's broadcast mechanism. The average-speed calculation is implemented as a Python UDF using pickup time, drop-off time, and trip distance.

The Ray implementation uses Ray Data and Pandas-based `map_batches` transformations. The zone lookup is materialized once as a small Pandas table and merged into each batch, giving a broadcast-style join without shuffling the full trip dataset.

There is one implementation detail worth documenting. A global duplicate-removal shuffle in Ray caused the local disk to reach the Ray spill threshold and the job failed with an `OutOfDiskError`. Inspection of the monthly files showed only 12 exact duplicate rows across all 32 files. For the successful benchmark, duplicate removal was therefore performed while preparing each monthly input file before the Ray Data pipeline. This avoids a very large full-dataset shuffle on the available 14 GiB machine.

The logical cleansing step is retained, but the duplicate-removal execution strategy is not byte-for-byte identical to Spark's global `dropDuplicates()`.

Benchmark Methodology

The benchmark was conducted on the same Ubuntu machine for both frameworks. Each framework was run once to completion using the configured two-worker cluster. System resource usage was observed with the `top` command during execution.

Total execution time covers the pipeline from ingestion through export. The Python UDF timer starts immediately before the average-speed transformation and is forced to execute fully before the UDF time is recorded.

The reported resource value is the peak %CPU shown for the busiest framework process in `top`. This is intentionally distinguished from the overall system CPU percentage because a multi-threaded process can exceed 100% by using multiple CPU cores.

Peak memory is reported as the maximum system memory in use during the recording.

Benchmark Results

Table 1: Measured performance of Spark and Ray.

Metric	Spark	Ray	Ray vs. Spark
Total execution time (s)	806.61	405.72	49.7% lower
Python UDF stage (s)	388.28	283.09	27.1% lower
Peak process CPU (%)	251.5	141.5	43.7% lower
Peak memory (MiB)	10,481.8	13,863.0	32.3% higher
Ray output rows	–	96,175,851	–

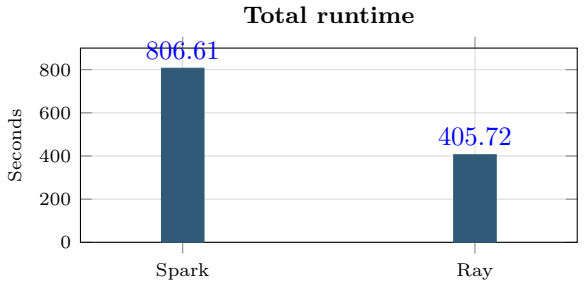
The most noticeable result is the difference in total wall-clock time. Ray completed the pipeline in 405.72 seconds, compared with 806.61 seconds for Spark. In this experiment, that is almost a twofold speedup.

The Python transformation also finished sooner in Ray: 283.09 seconds versus 388.28 seconds in Spark.

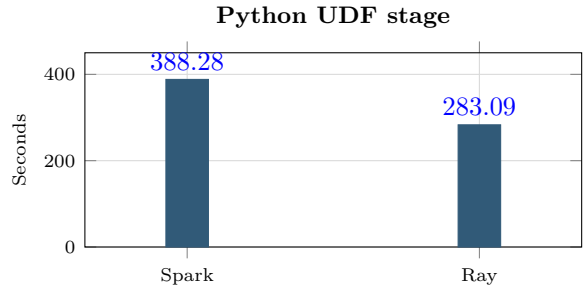
The resource measurements show a trade-off rather than a universal win. The busiest Spark process reached a higher peak CPU percentage, while Ray reached a higher peak memory level. In other words, Ray was faster in this run, but it also pushed the available memory more aggressively.

Performance graphs

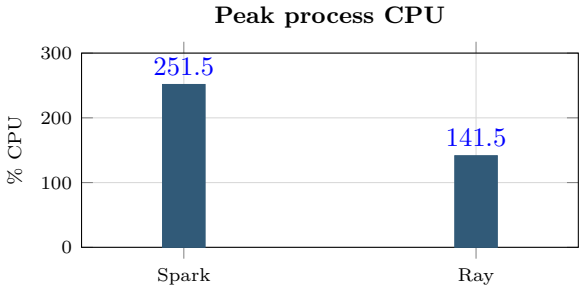
The figures below are generated directly from the measured values in Table 1; no estimated benchmark values are used.



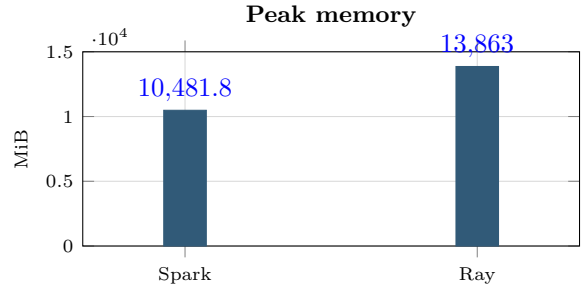
(a) Total execution time.



(b) UDF stage time.



(c) Busiest framework process.



(d) Peak system memory.

Figure 2: Visual comparison of the measured performance metrics.

Python UDF Deep-Dive

The UDF experiment is useful because it isolates a part of the workload where the execution model of the two frameworks becomes especially visible.

Spark uses a Python UDF inside a Spark dataframe plan, which introduces work across Spark's JVM-oriented execution system and the Python runtime. Ray Data, on the other hand, applies the average-speed function directly inside Python batches.

The measured UDF stage was 388.28 seconds in Spark and 283.09 seconds in Ray, a difference of 105.19 seconds.

The result is consistent with the intuition that a Python-native execution path can be attractive when a workload is already dominated by Python-side transformations.

At the same time, the benchmark is a single experiment on one machine, so this result should be interpreted as evidence for this workload rather than a claim that Ray's UDF performance is always superior.

One interesting point is that the Ray UDF measurement includes the upstream Ray Data operations required to feed the transformation, because Ray Data executes lazily and the benchmark forces the pipeline to materialize through the aggregation call.

This makes the timing useful as an end-to-end UDF-stage measurement, but not a microbenchmark of the Python function alone.

Discussion: Which Framework Wins?

For the workload used here, Ray is the clear winner on execution time. Its 405.72-second runtime is 49.7% lower than Spark's 806.61 seconds, and its measured Python UDF stage is also 27.1% lower.

The resource results make the comparison more balanced. Spark's busiest process reached a higher peak process CPU value, while Ray reached about 13.9 GiB of system memory in use.

On a machine with only about 14 GiB of RAM, that is a meaningful constraint. The faster framework is therefore not automatically the safer choice on a memory-constrained machine.

For an **AI-first project**, I would choose **Ray** for this experiment because the workload contains a substantial Python transformation and Ray delivered the better observed runtime.

For a **BI-first project**, I would choose **Spark**, mainly because the task is expressed naturally as structured dataframe operations and Spark is a mature choice for conventional batch analytics and SQL-oriented processing.

The benchmark does not imply that Spark is unsuitable for AI or that Ray is unsuitable for BI; the recommendation follows the workload characteristics and the observed measurements.

Performance Tuning Note

AI assistance was used during development to translate Spark-style dataframe logic into Ray Data operations and to reason about the cost of a global duplicate-removal shuffle.

One important tuning change was to avoid the full Ray hash shuffle for duplicate removal after it caused disk exhaustion on the available machine.

The successful Ray run instead deduplicated each monthly input file before distributed processing. This reduced the storage pressure substantially, while the file inspection showed only 12 exact duplicate rows across the full collection of monthly files.

Conclusion

The benchmark shows a clear time advantage for Ray on this Python-heavy ETL workload.

Ray completed the pipeline in 405.72 seconds, approximately half the time taken by Spark, and its measured Python UDF stage was also shorter. The main cost of this advantage was higher peak memory usage.

Therefore, the winner for this particular benchmark is **Ray**.

The result should be understood as a workload-specific comparison: on this machine and with this Python/UDF-

heavy preprocessing pipeline, Ray provided the better overall runtime, while Spark remained competitive in terms of memory pressure and remains a strong choice for BI-oriented structured-data processing.

Reproducibility and Evidence

The following files/screenshots should accompany the submitted PDF and code repository:

- Spark Master UI showing the two active Spark workers.
- Ray Dashboard cluster/resource view showing the Ray head and two workers.
- The two completed benchmark terminal outputs.
- `spark_clean.py` and `ray_clean.py` in the code repository.